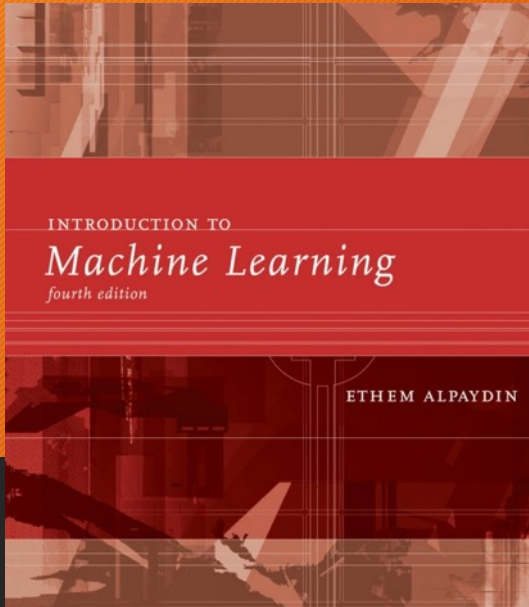




Lecture Slides for

INTRODUCTION TO Machine Learning

4e

ETHEM ALPAYDIN
The MIT Press, 2020

ethem.alpaydin@gmail.com

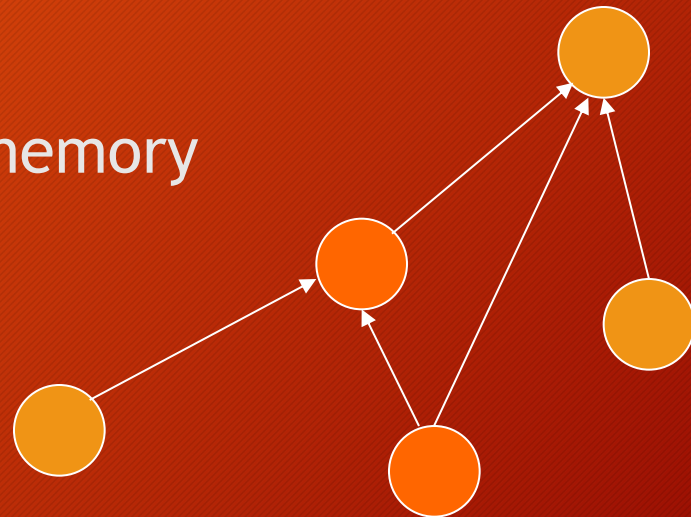
CHAPTER 11:

Multilayer Perceptrons

Neural Networks

3

- Networks of processing units (neurons) with connections (synapses) between them
- Large number of neurons: 10^{10}
- Large connectivity: 10^5
- Parallel processing
- Distributed computation/memory
- Robust to noise, failures



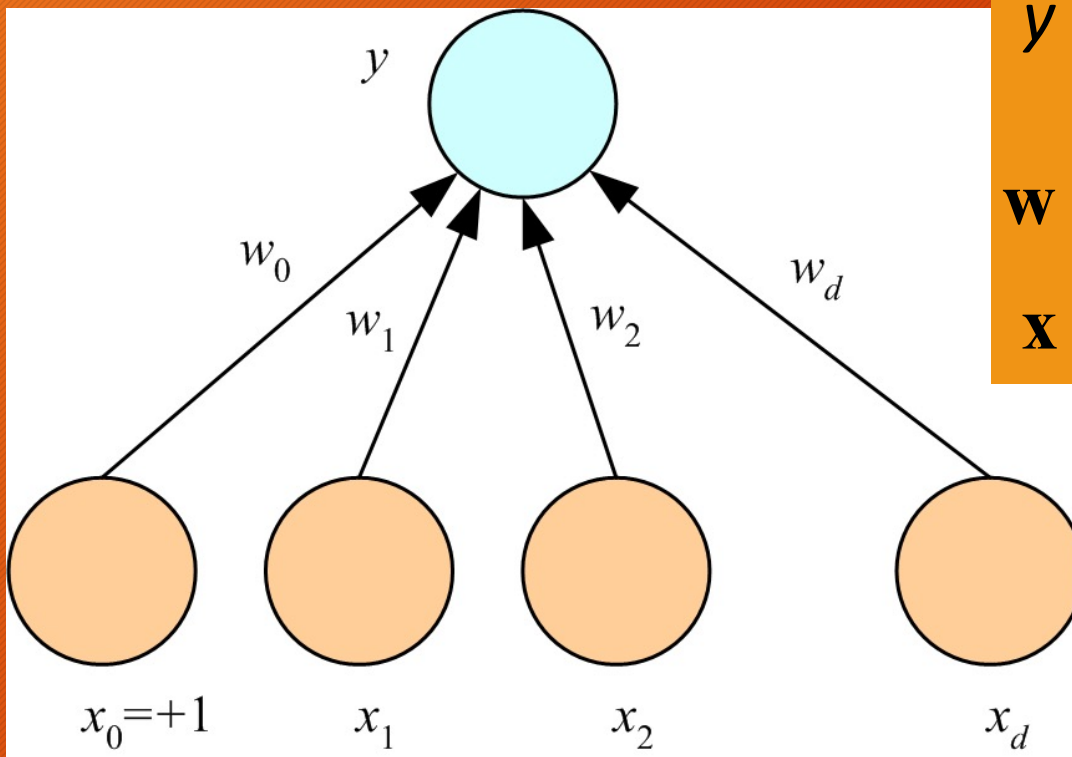
Understanding the Brain

4

- Levels of analysis (Marr, 1982)
 1. Computational theory
 2. Representation and algorithm
 3. Hardware implementation
- Reverse engineering: From hardware to theory
- Parallel processing: SIMD vs MIMD
 - Neural net: SIMD with modifiable local memory
 - Learning: Update by training/experience

Perceptron

5



$$y = \sum_{j=1}^d w_j x_j + w_0 = \mathbf{w}^T \mathbf{x}$$

$$\mathbf{w} = [w_0, w_1, \dots, w_d]^T$$

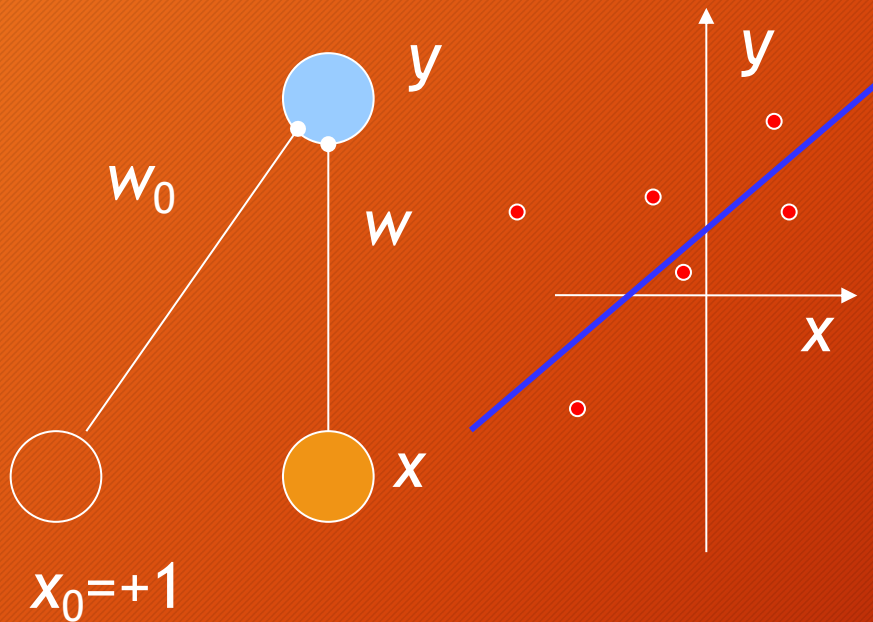
$$\mathbf{x} = [1, x_1, \dots, x_d]^T$$

(Rosenblatt, 1962)

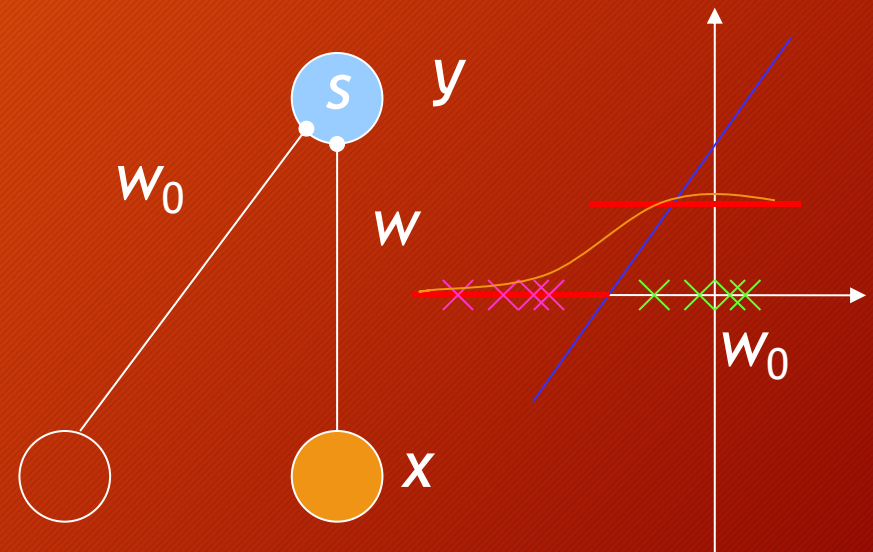
What a Perceptron Does

6

- Regression: $y = wx + w_0$



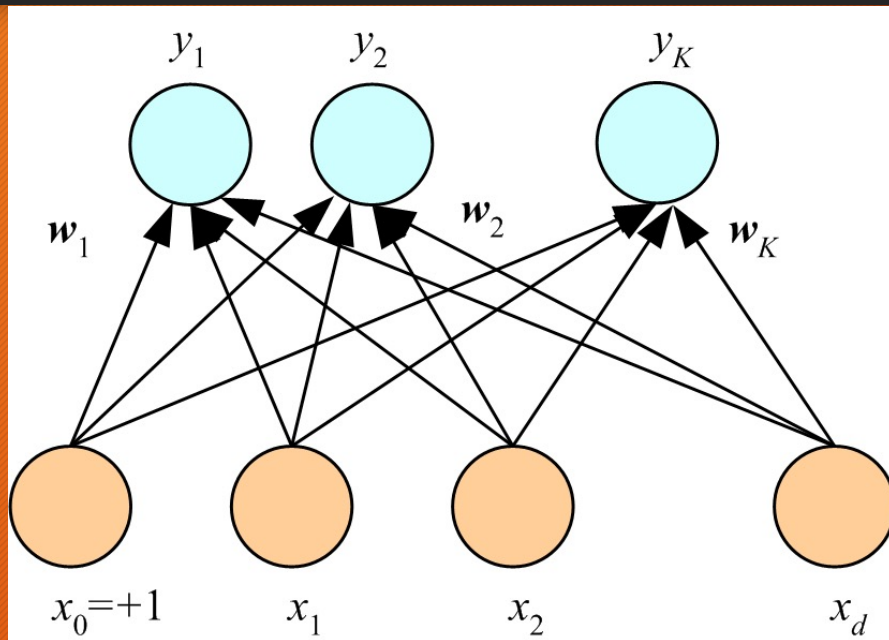
- Classification: $y = 1 (wx + w_0 > 0)$



$$y = \text{sigmoid}(o) = \frac{1}{1 + \exp[-w^T \mathbf{x}]}$$

K Outputs

7



Regression:

$$y_i = \sum_{j=1}^d w_{ij} x_j + w_{i0} = \mathbf{w}_i^T \mathbf{x}$$

$$\mathbf{y} = \mathbf{W}\mathbf{x}$$

Classification:

$$o_i = \mathbf{w}_i^T \mathbf{x}$$

$$y_i = \frac{\exp o_i}{\sum_k \exp o_k}$$

choose C_i

$$\text{if } y_i = \max_k y_k$$

Training

8

- Online (instances seen one by one) vs batch (whole sample) learning:
 - No need to store the whole sample
 - Problem may change in time
 - Wear and degradation in system components
- Stochastic gradient-descent: Update after a single pattern
- Generic update rule (LMS rule):

$$\Delta w_{ij}^t = \eta (r_i^t - y_i^t) x_j^t$$

Update=LearningFactor·(DesiredOutput–ActualOutput)·Input

Training a Perceptron: Regression

9

- Regression (Linear output):

$$E^t(\mathbf{w} \mid \mathbf{x}^t, r^t) = \frac{1}{2} (r^t - y^t)^2 = \frac{1}{2} [r^t - (\mathbf{w}^T \mathbf{x}^t)]^2$$

$$\Delta \mathbf{w}_j^t = \eta (r^t - y^t) \mathbf{x}_j^t$$

Classification

10

- Single sigmoid output

$$y^t = \text{sigmoid}(\mathbf{w}^T \mathbf{x}^t)$$

$$E^t(\mathbf{w} \mid \mathbf{x}^t, \mathbf{r}^t) = -r^t \log y^t - (1 - r^t) \log (1 - y^t)$$

$$\Delta \mathbf{w}_j^t = \eta (r^t - y^t) x_j^t$$

- $K > 2$ softmax outputs

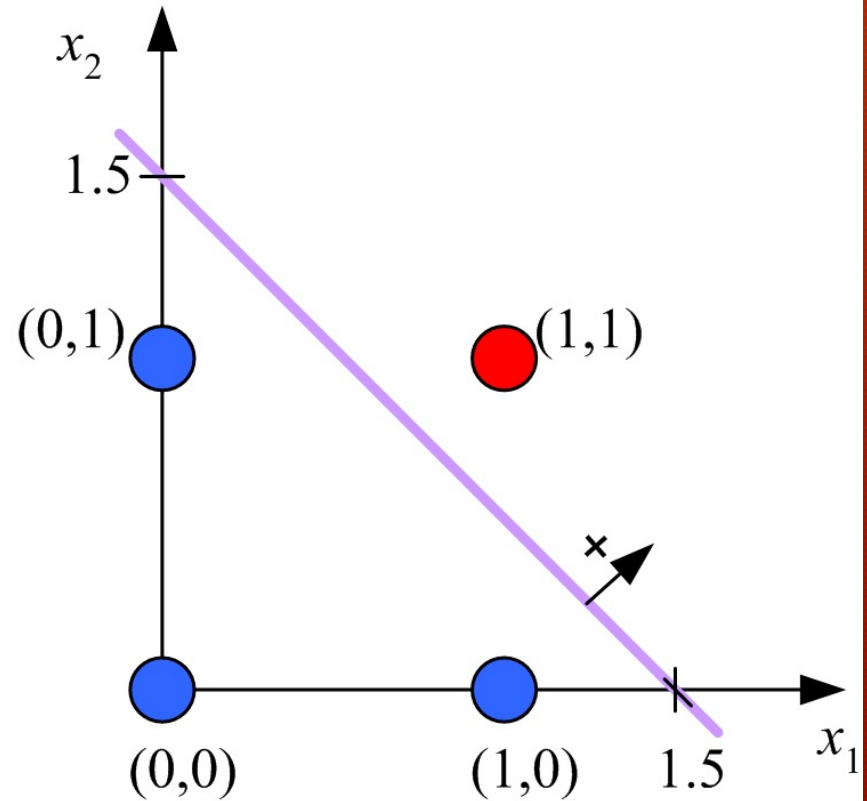
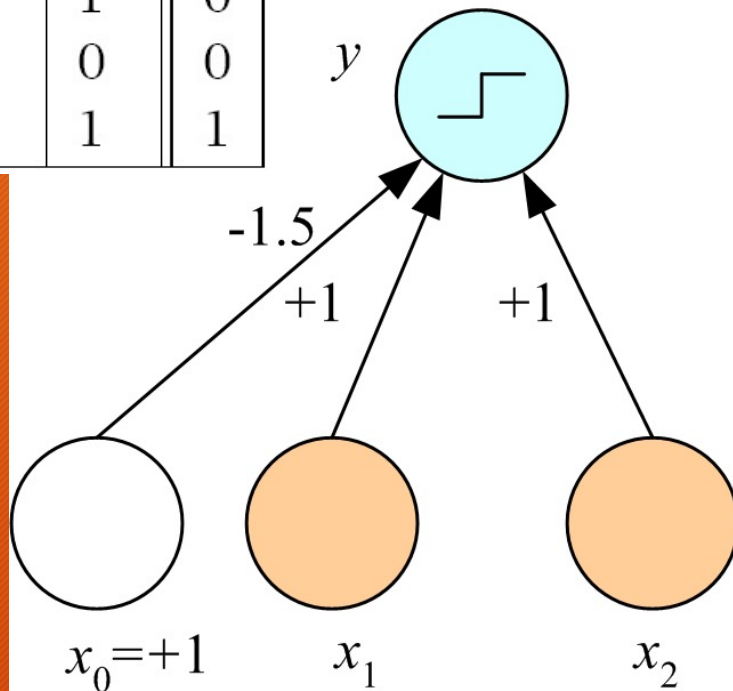
$$y^t = \frac{\exp \mathbf{w}_i^T \mathbf{x}^t}{\sum_k \exp \mathbf{w}_k^T \mathbf{x}^t} \quad E^t(\{\mathbf{w}_i\}_i \mid \mathbf{x}^t, \mathbf{r}^t) = -\sum_i r_i^t \log y_i^t$$

$$\Delta \mathbf{w}_{ij}^t = \eta (r_i^t - y_i^t) x_j^t$$

Learning Boolean AND

11

x_1	x_2	r
0	0	0
0	1	0
1	0	0
1	1	1



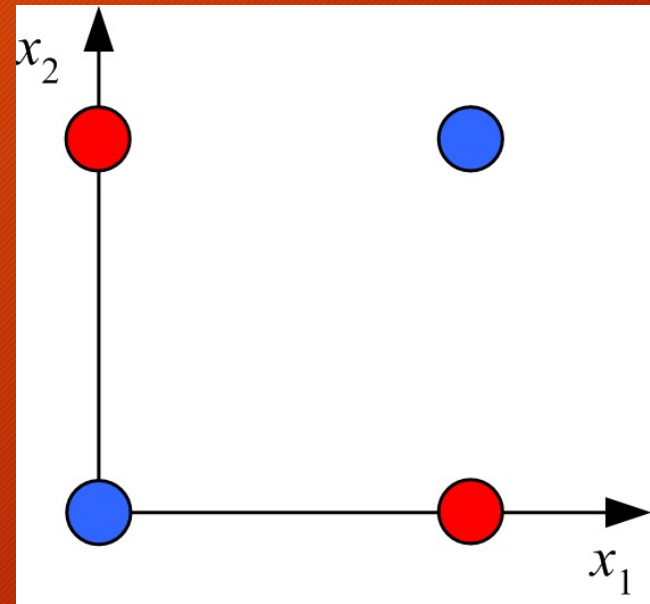
XOR

12

x_1	x_2	r
0	0	0
0	1	1
1	0	1
1	1	0

- No w_0, w_1, w_2 satisfy:

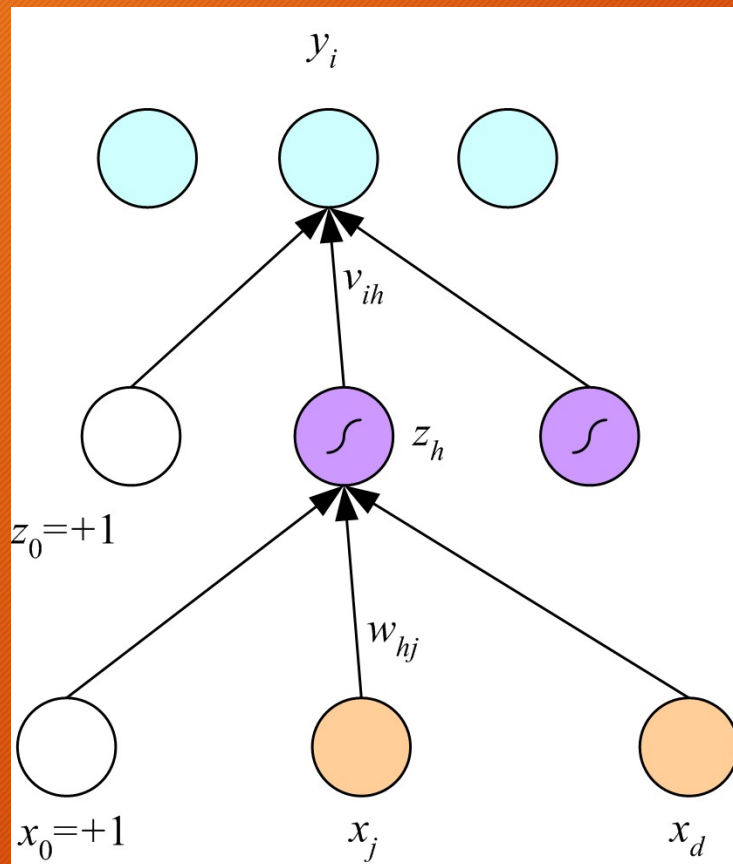
$$\begin{array}{rcl} & w_0 & \leq 0 \\ w_2 + & w_0 & > 0 \\ w_1 + & w_0 & > 0 \\ w_1 + & w_2 + & w_0 \leq 0 \end{array}$$



(Minsky and Papert, 1969)

Multilayer Perceptrons

13

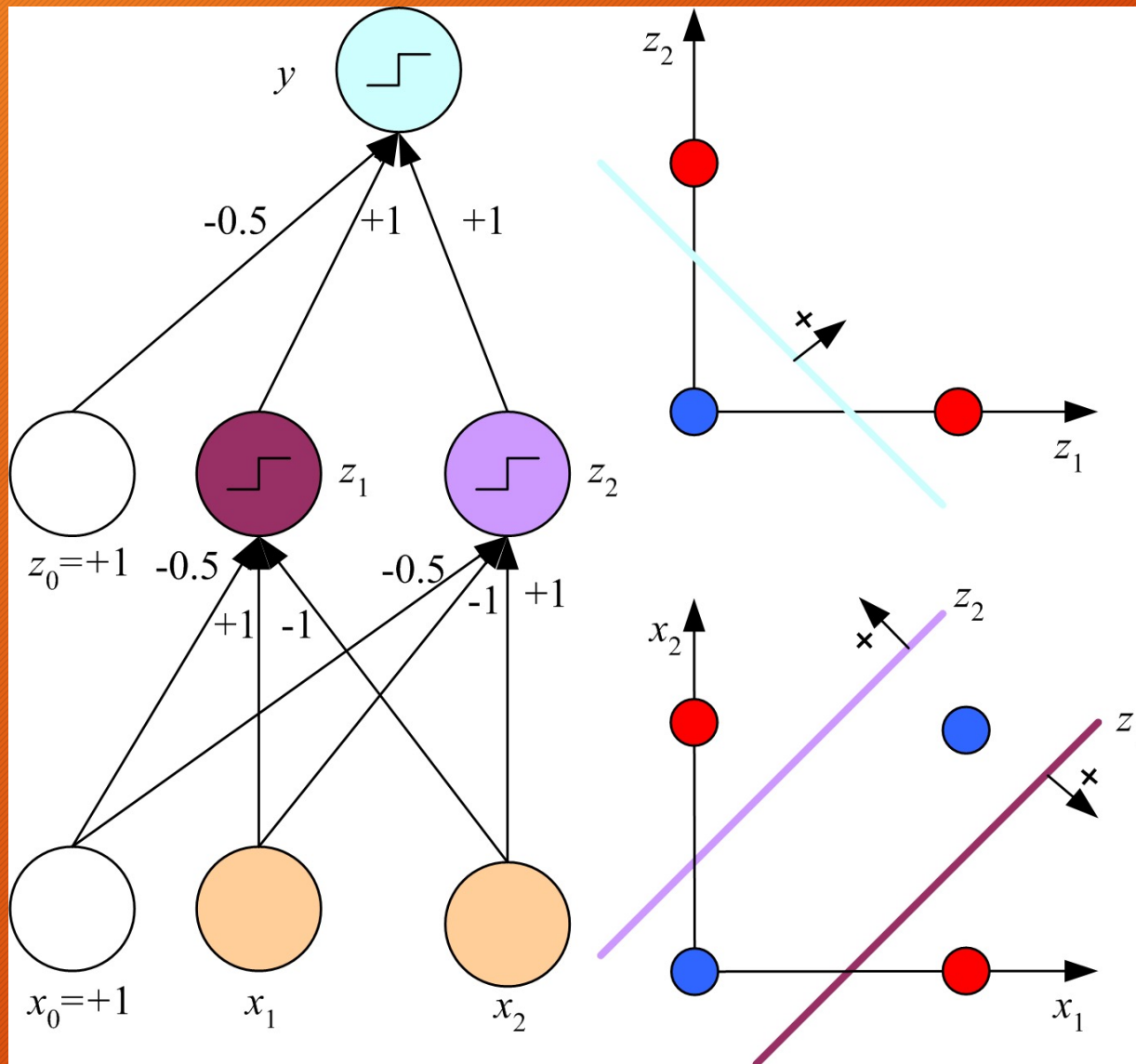


$$y_i = \mathbf{v}_i^T \mathbf{z} = \sum_{h=1}^H v_{ih} z_h + v_{i0}$$

$$z_h = \text{sigmoid}(\mathbf{w}_h^T \mathbf{x})$$

$$= \frac{1}{1 + \exp\left[-\left(\sum_{j=1}^d w_{hj} x_j + w_{h0}\right)\right]}$$

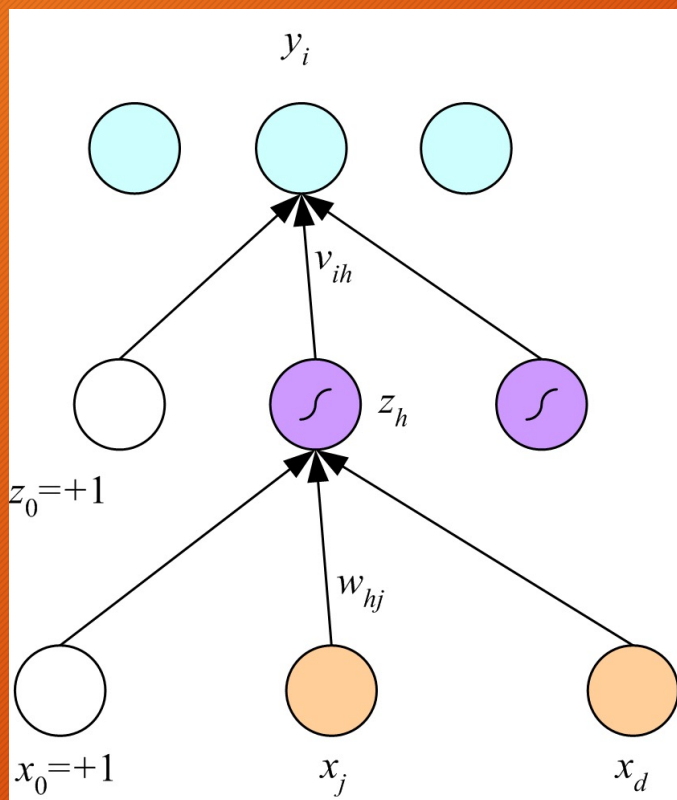
(Rumelhart et al., 1986)



$$x_1 \text{ XOR } x_2 = (x_1 \text{ AND } \sim x_2) \text{ OR } (\sim x_1 \text{ AND } x_2)$$

Backpropagation

15



$$y_i = \mathbf{v}_i^T \mathbf{z} = \sum_{h=1}^H v_{ih} z_h + v_{i0}$$

$$z_h = \text{sigmoid}(\mathbf{w}_h^T \mathbf{x})$$

$$= \frac{1}{1 + \exp\left[-\left(\sum_{j=1}^d w_{hj} x_j + w_{h0}\right)\right]}$$

$$\frac{\partial E}{\partial w_{hj}} = \frac{\partial E}{\partial y_i} \frac{\partial y_i}{\partial z_h} \frac{\partial z_h}{\partial w_{hj}}$$

Regression

16

$$E(\mathbf{W}, \mathbf{v} | \mathcal{X}) = \frac{1}{2} \sum_t (r^t - y^t)^2$$

$$y^t = \sum_{h=1}^H v_h z_h^t + v_0$$

Forward

$$z_h = \text{sigmoid}(\mathbf{w}_h^T \mathbf{x})$$

$\mathbf{x}$

$$\Delta v_h = \sum_t (r^t - y^t) z_h^t$$

Backward

$$\begin{aligned} \Delta w_{hj} &= -\eta \frac{\partial E}{\partial w_{hj}} \\ &= -\eta \sum_t \frac{\partial E}{\partial y^t} \frac{\partial y^t}{\partial z_h^t} \frac{\partial z_h^t}{\partial w_{hj}} \\ &= -\eta \sum_t \boxed{-(r^t - y^t)} v_h z_h^t (1 - z_h^t) x_j^t \\ &= \eta \sum_t (r^t - y^t) v_h z_h^t (1 - z_h^t) x_j^t \end{aligned}$$

Regression with Multiple Outputs

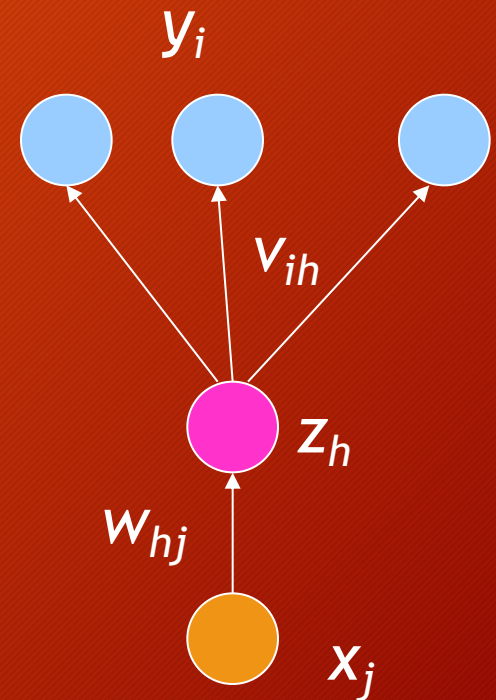
17

$$E(\mathbf{W}, \mathbf{V} | \mathcal{X}) = \frac{1}{2} \sum_t \sum_i (r_i^t - y_i^t)^2$$

$$y_i^t = \sum_{h=1}^H v_{ih} z_h^t + v_{i0}$$

$$\Delta v_{ih} = \eta \sum_t (r_i^t - y_i^t) z_h^t$$

$$\Delta w_{hj} = \eta \sum_t \left[\sum_i (r_i^t - y_i^t) v_{ih} \right] z_h^t (1 - z_h^t) x_j^t$$



Initialize all v_{ih} and w_{hj} to $\text{rand}(-0.01, 0.01)$

Repeat

For all $(\mathbf{x}^t, r^t) \in \mathcal{X}$ in random order

For $h = 1, \dots, H$

$$z_h \leftarrow \text{sigmoid}(\mathbf{w}_h^T \mathbf{x}^t)$$

For $i = 1, \dots, K$

$$y_i = \mathbf{v}_i^T \mathbf{z}$$

For $i = 1, \dots, K$

$$\Delta \mathbf{v}_i = \eta(r_i^t - y_i^t) \mathbf{z}$$

For $h = 1, \dots, H$

$$\Delta \mathbf{w}_h = \eta\left(\sum_i (r_i^t - y_i^t) v_{ih}\right) z_h (1 - z_h) \mathbf{x}^t$$

For $i = 1, \dots, K$

$$\mathbf{v}_i \leftarrow \mathbf{v}_i + \Delta \mathbf{v}_i$$

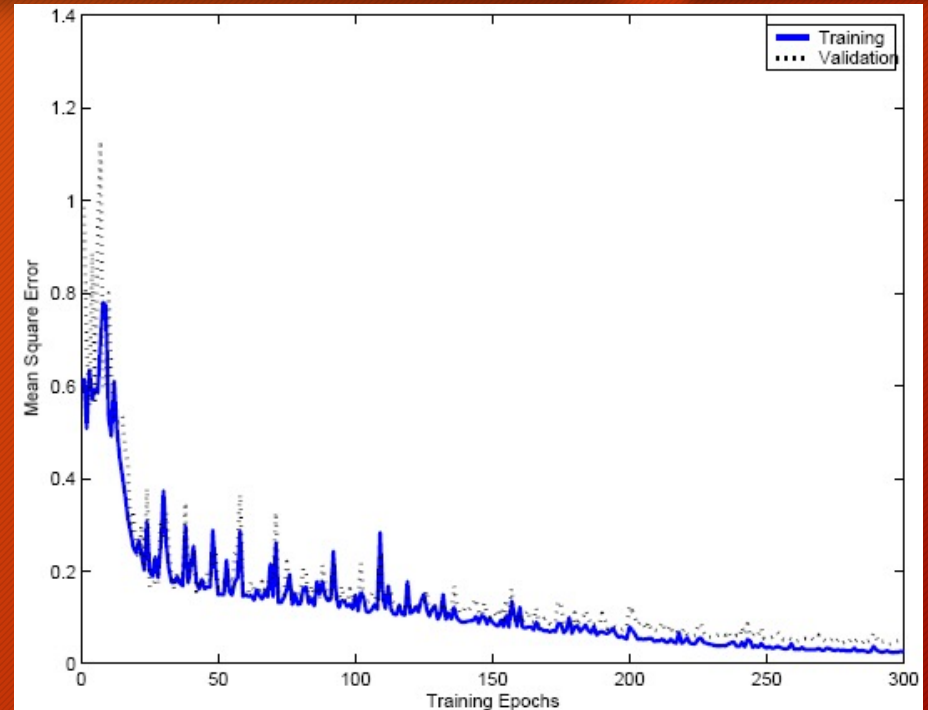
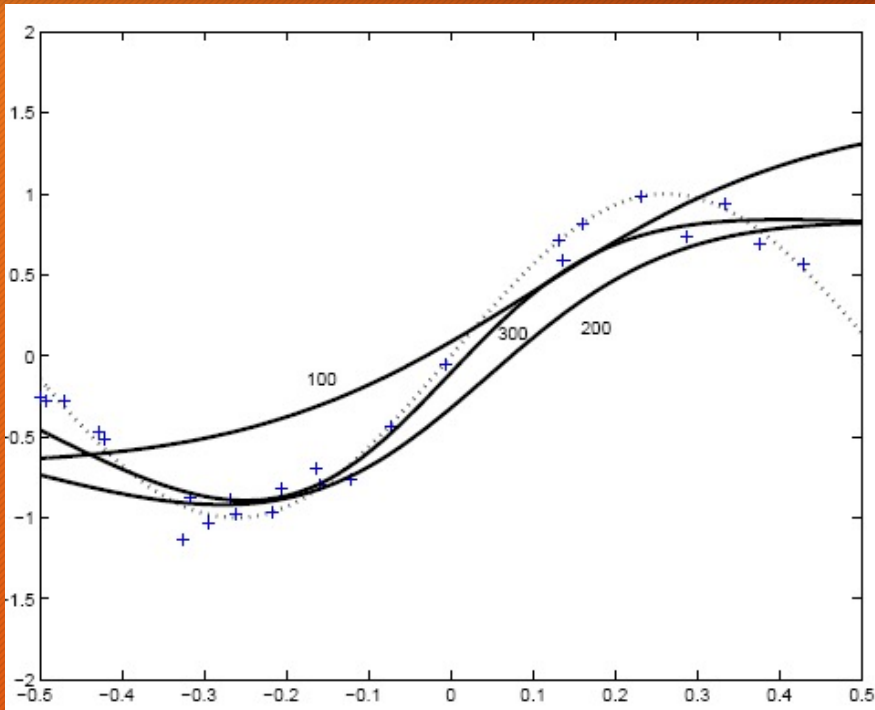
For $h = 1, \dots, H$

$$\mathbf{w}_h \leftarrow \mathbf{w}_h + \Delta \mathbf{w}_h$$

Until convergence

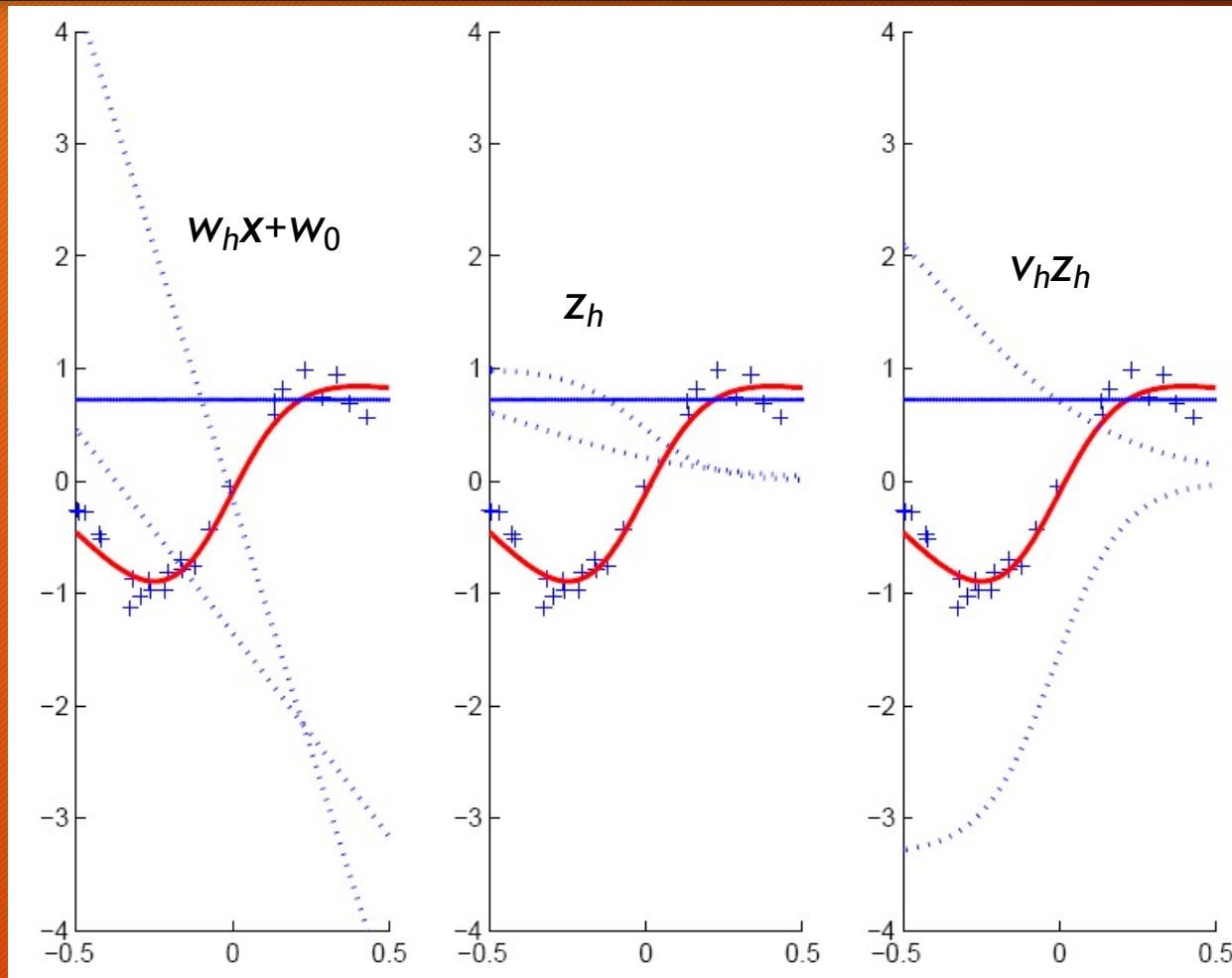
1d Regression: Convergence

19



1d Regression: What hidden units do

20



Two-Class Discrimination

21

- One sigmoid output y^t for $P(C_1 | \mathbf{x}^t)$ and $P(C_2 | \mathbf{x}^t) \equiv 1 - y^t$

$$y^t = \text{sigmoid} \left(\sum_{h=1}^H v_h z_h^t + v_0 \right)$$

$$E(\mathbf{W}, \mathbf{v} | \mathcal{X}) = - \sum_t r^t \log y^t + (1 - r^t) \log (1 - y^t)$$

$$\Delta v_h = \eta \sum_t (r^t - y^t) z_h^t$$

$$\Delta w_{hj} = \eta \sum_t (r^t - y^t) v_h z_h^t (1 - z_h^t) x_j^t$$

$K > 2$ Classes

22

$$o_i^t = \sum_{h=1}^H v_{ih} z_h^t + v_{i0} \quad y_i^t = \frac{\exp o_i^t}{\sum_k \exp o_k^t} \equiv P(C_i | \mathbf{x}^t)$$

$$E(\mathbf{W}, \mathbf{v} | \mathcal{X}) = - \sum_t \sum_i r_i^t \log y_i^t$$

$$\Delta v_{ih} = \eta \sum_t (r_i^t - y_i^t) z_h^t$$

$$\Delta w_{hj} = \eta \sum_t \left[\sum_i (r_i^t - y_i^t) v_{ih} \right] z_h^t (1 - z_h^t) x_j^t$$

Multiple Labels

23

- K labels independently 0 or 1

$$r_i^t = \begin{cases} 1 & \text{if } \mathbf{x}^t \text{ has label } i \\ 0 & \text{otherwise} \end{cases}$$

- Sigmoid outputs

$$y_i^t = \text{sigmoid} \left(\sum_{h=1}^H v_{ih} z_h^t + v_{i0} \right)$$

- We minimize sum of cross-entropy losses

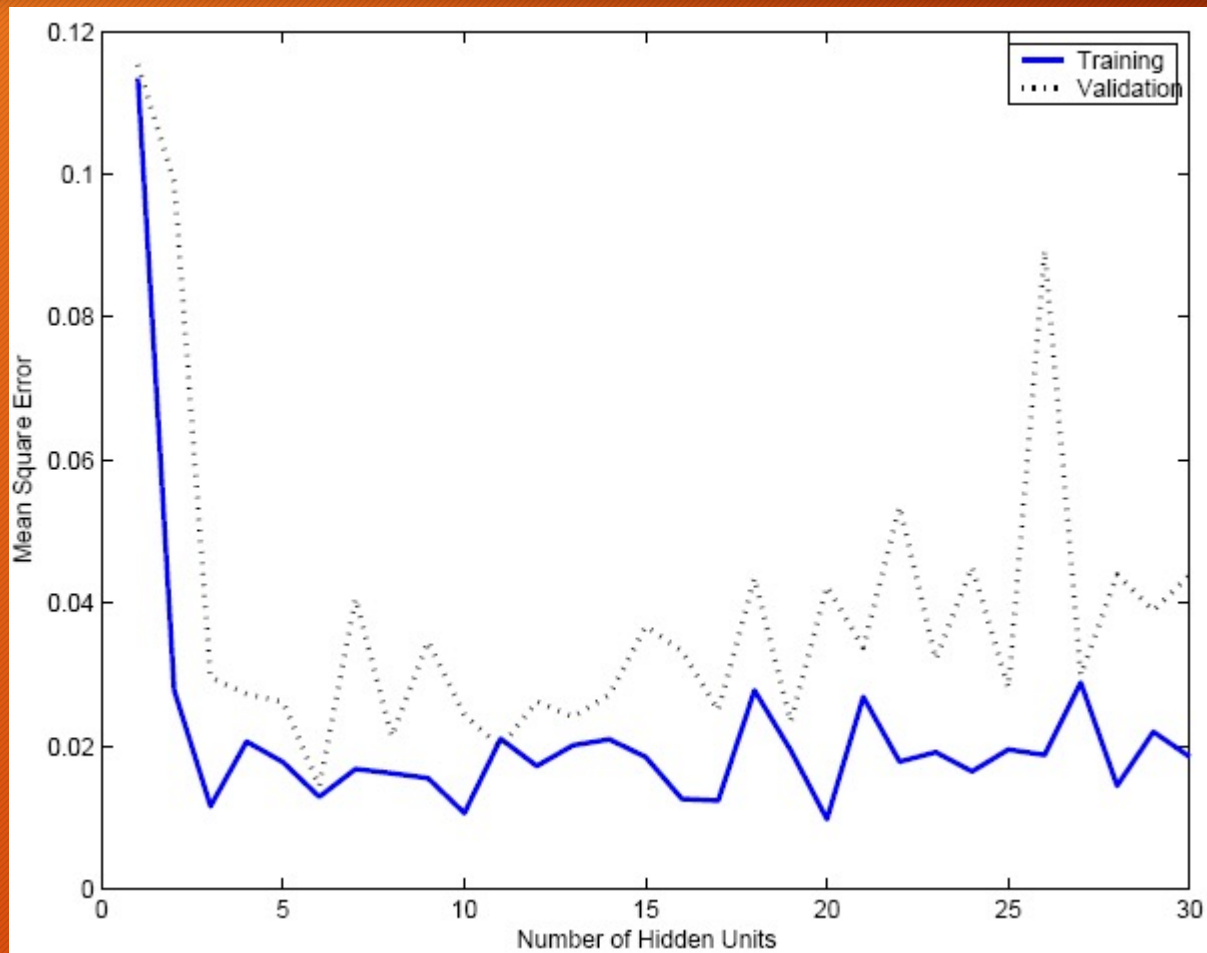
$$E(\mathbf{W}, \mathbf{V} | \mathcal{X}) = - \sum_t \sum_i r_i^t \log y_i^t + (1 - r_i^t) \log(1 - y_i^t)$$

$$\Delta v_{ih} = \eta \sum_t (r_i^t - y_i^t) z_h^t$$

$$\Delta w_{hj} = \eta \sum_t \left[\sum_i (r_i^t - y_i^t) v_{ih} \right] z_h^t (1 - z_h^t) x_j^t$$

Overfitting

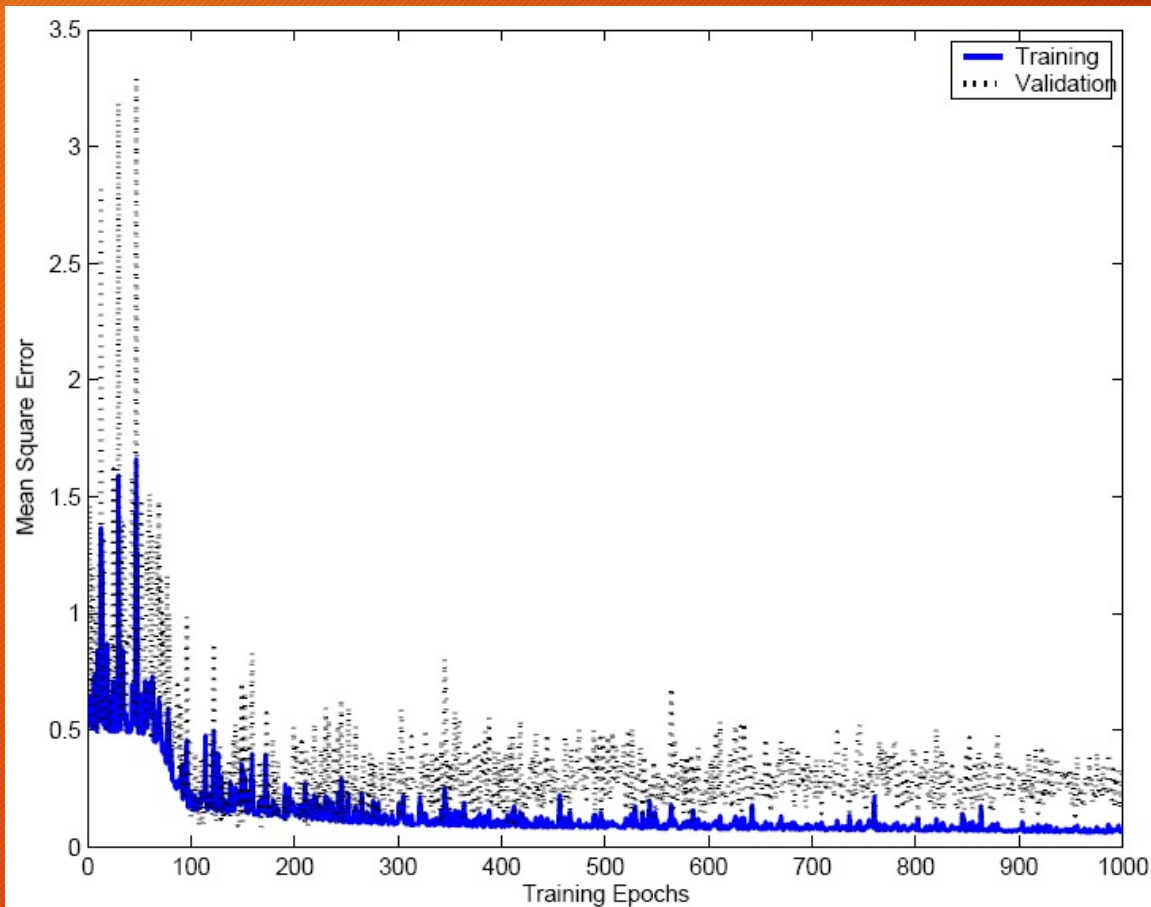
24



Number of weights:
 $H(d+1) + (H+1)K$

Overtraining

25



Learning Hidden Representations

26

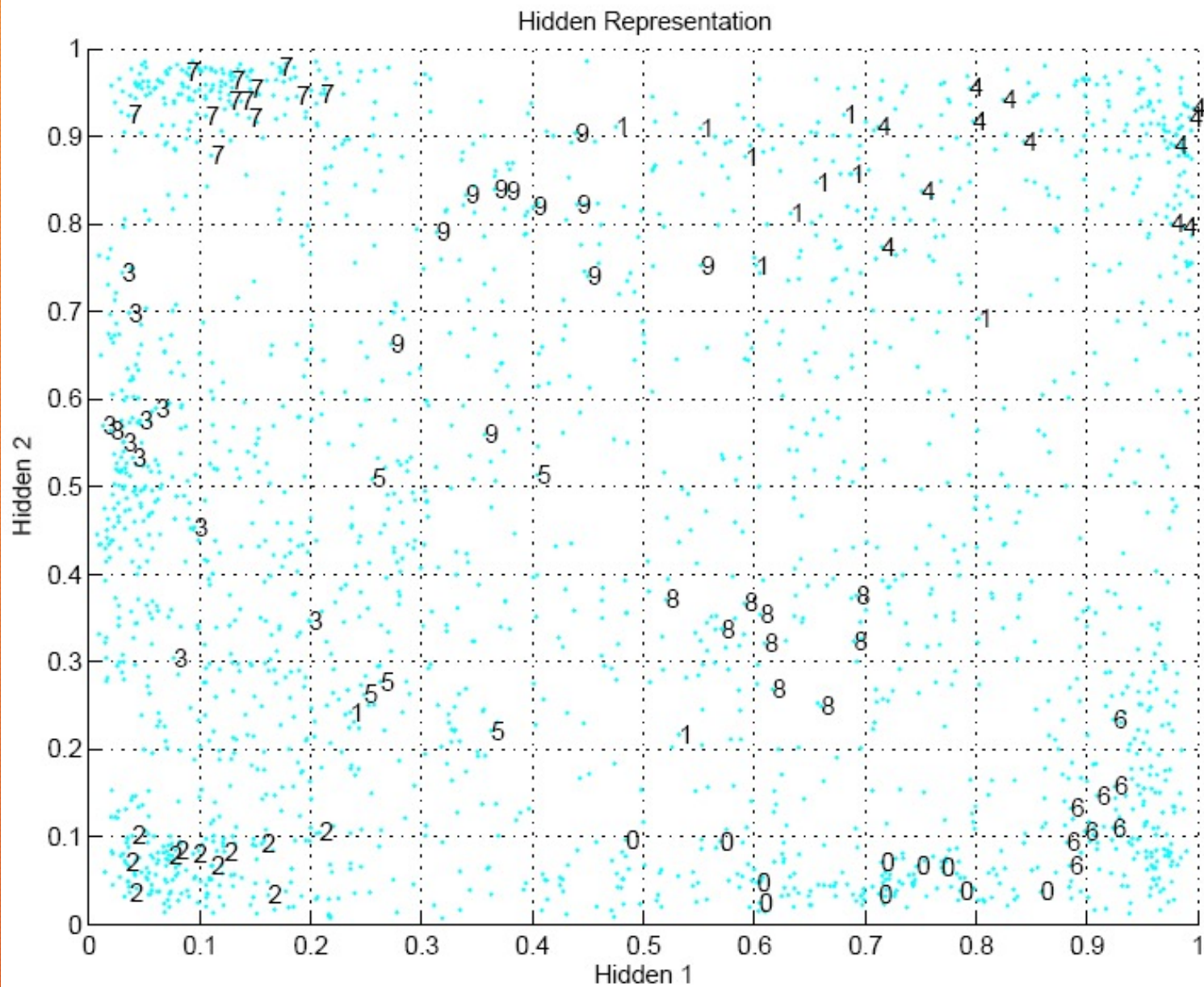
- MLP is a generalized linear model where hidden units are the nonlinear basis functions:

$$y = \sum_{h=1}^H v_h \phi(\mathbf{x} | \mathbf{w}_h)$$

where

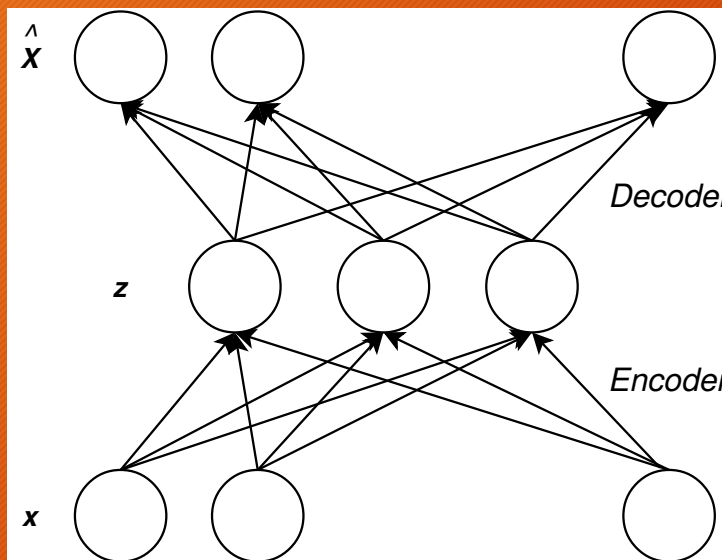
$$\phi(\mathbf{x} | \mathbf{w}_h) \equiv \text{sigmoid}(\mathbf{w}_h^T \mathbf{x})$$

- The advantage is that the basis function parameters can also be learned from data.
- The hidden units, z_h , learn a *code/embedding*, a representation in the hidden space
- **Transfer learning**: Use code in another task
- Semisupervised learning: Transfer from an unsupervised to supervised problem



Autoencoders

28



$$\hat{\mathbf{x}}^t = \text{Dec}(\mathbf{z}^t | \mathbf{V})$$

$$\mathbf{z}^t = \text{Enc}(\mathbf{x}^t | \mathbf{W})$$

$$E(\mathbf{W}, \mathbf{V} | \mathcal{X}) = \sum_t \|\mathbf{x}^t - \hat{\mathbf{x}}^t\|^2 = \sum_t \|\mathbf{x}^t - \text{Dec}(\text{Enc}(\mathbf{x}^t | \mathbf{W}) | \mathbf{V})\|^2$$

- Variants: Denoising, sparse autoencoders

Word2vec

29

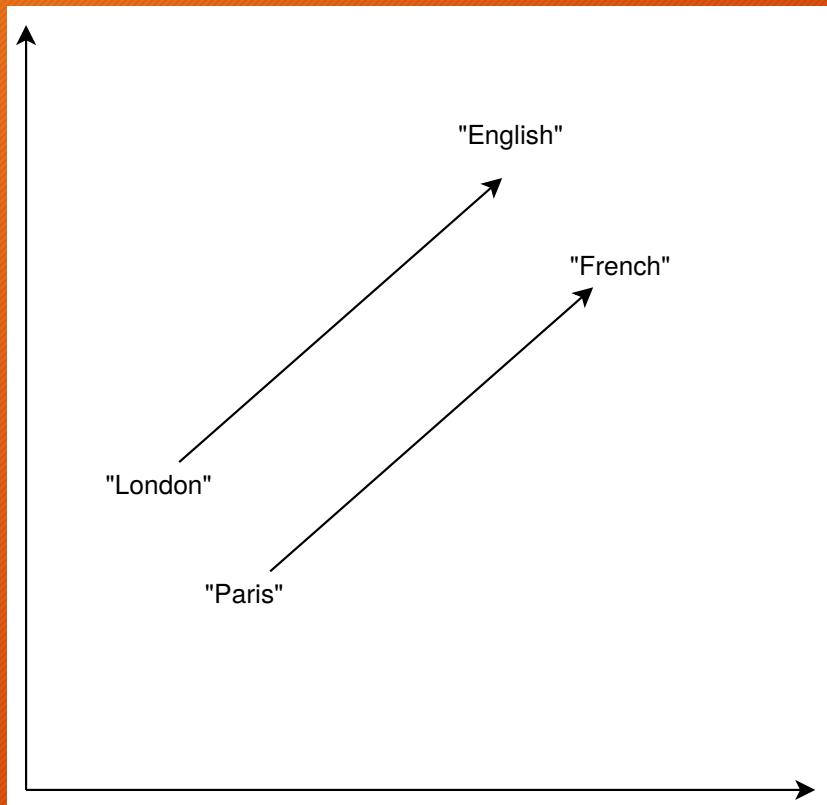
- Learn an embedding for words for NLP
- Skipgram: An autencoder with linear encoder and decoder where input is the center word and output is a context word



- Similar words appear in similar contexts, so similar codes will be learned for them

Vector Algebra

30



Because they will always appear in similar contexts in pairs in a large corpus, we expect

$\text{vec}(\text{"English"}) - \text{vec}(\text{"London"})$

to be similar to

$\text{vec}(\text{"French"}) - \text{vec}(\text{"Paris"})$

so,

$\text{vec}(\text{"Paris"}) + \text{vec}(\text{"English"}) - \text{vec}(\text{"London"})$

will be similar to

$\text{vec}(\text{"French"})$